

📖 Library Management System – Enterprise Architecture Technical Report

(Full Rewrite – Same Structure, More Professional & Technical)

Author: Mahdy Mokhtari

Role: Backend Engineer (Java / Spring Boot)

Repository: MahdyMokh7/library-management-api

1. Executive Summary

The Library Management System is a production-grade backend platform engineered using **Java 22**, **Spring Boot 3**, **Spring Data JPA**, **Hibernate**, **PostgreSQL**, **Flyway**, **MapStruct**, and a comprehensive DevOps pipeline leveraging **Docker**, **GitHub Actions**, **Spotless**, **PMD**, **JaCoCo**, and **Testcontainers**.

The system implements the full lifecycle of library operations, including book registration, catalog browsing, advanced search, borrowing, returning, and soft deletion. It is designed with **Clean Architecture**, **SOLID principles**, and **Domain-Driven Design (DDD) influences**, ensuring high maintainability, scalability, and clarity of responsibilities.

This report consolidates all project artifacts – the official documentation, the README, and handwritten Persian design notes – into a unified, deeply technical, enterprise-grade architectural document suitable for senior engineering review.

2. System Overview

2.1 Business Domain

The system models a real-world library with two core business capabilities:

Book Management

- Register new books
- Retrieve complete book catalog
- Apply pagination, sorting, and multi-criteria filtering
- Update book metadata (ISBN immutable)
- Soft delete books (only if not currently borrowed)

Borrowing Management

- Borrow available books
- Return borrowed books
- Track borrowing history by book or borrower
- Enforce business rules (availability, return constraints, soft delete restrictions)

These capabilities are separated into distinct controllers and services, reflecting domain boundaries and aligning with enterprise design patterns.

2.2 Architectural Style

The system follows a **Layered Architecture**, providing clear separation of concerns:

```
Presentation Layer (Controllers)
↓
Application Layer (Services, Validators)
↓
Domain Layer (Entities, Business Rules)
↓
Infrastructure Layer (Repositories, Database, ORM)
```

This architecture ensures:

- High maintainability
- Strong testability
- Clear responsibility boundaries
- Predictable scalability
- Alignment with enterprise backend standards

Alternatives such as Hexagonal Architecture or CQRS were considered but deemed unnecessary for the scope and complexity of this project.

2.3 Technology Stack

Component	Purpose
Java 22	Modern language features, performance, virtual threads readiness
Spring Boot 3	Application framework, auto-configuration
Spring Data JPA	Repository abstraction, query generation
Hibernate	ORM implementation
PostgreSQL	Production-grade relational database
Flyway	Versioned database migrations
MapStruct	Compile-time DTO/entity mapping
Lombok	Boilerplate reduction
JUnit 5 + Mockito	Unit testing
Testcontainers	Real PostgreSQL integration tests
JaCoCo	Coverage reporting
PMD	Static code analysis
Spotless	Code formatting enforcement
Docker	Containerization
GitHub Actions	CI/CD pipeline

This stack reflects industry best practices for building robust, maintainable, and scalable backend systems.

3. Detailed System Explanation

3.1 Domain Model

Book Entity

Represents a single physical book instance.

Fields:

- title
- author
- isbn
- publicationYear
- available
- deleted

Design Rationale:

- Each book is modeled as an independent entity rather than using a “copies count”.
- This enables precise borrowing history, supports future item types (Magazine, Ebook, AudioBook), and avoids ambiguity in concurrent borrowing scenarios.
- Aligns with DDD’s concept of an Entity with a unique identity and lifecycle.

BorrowingRecord Entity

Represents a complete borrowing lifecycle.

Fields:

- book
- borrowerName
- borrowDate
- returnDate
- status

Design Rationale:

- A single record captures both borrow and return events.
- Simplifies lifecycle tracking, preserves audit trails, and supports future analytics.
- Avoids redundant “borrow” and “return” records.

3.2 API Layer

BookController

Handles CRUD operations and search.

Endpoints:

- POST /api/v1/books
- GET /api/v1/books
- GET /api/v1/books/{id}
- PUT /api/v1/books/{id}
- DELETE /api/v1/books/{id}

BorrowingController

Handles borrowing lifecycle.

Endpoints:

- POST /api/v1/borrowings/borrow
- POST /api/v1/borrowings/return
- GET /api/v1/borrowings/book/{id}
- GET /api/v1/borrowings/borrower/{name}

Rationale:

- Clear domain separation improves maintainability, readability, and testability.
- Aligns with REST best practices and Swagger documentation grouping.

3.3 Service Layer

The service layer encapsulates all business rules.

BookService Responsibilities

- Validate book creation
- Enforce ISBN immutability
- Prevent deletion of borrowed books
- Apply soft delete
- Compose dynamic search specifications

BorrowingService Responsibilities

- Validate borrowing eligibility
- Update book availability
- Create borrowing record
- Update record on return
- Prevent returning non-borrowed books

SOLID Principles Applied:

- **SRP:** Each service handles one domain.
- **OCP:** New features added without modifying existing logic.
- **DIP:** Services depend on repository abstractions.

3.4 Validation Layer

Uses **Jakarta Bean Validation** for:

- Required fields
- String constraints
- Publication year validation
- Borrowing request validation
- Return request validation

Ensures data integrity and prevents invalid states from entering the system.

3.5 Exception Handling

A **GlobalExceptionHandler** provides:

- Unified error responses
- Structured validation error formatting
- Business rule violation handling
- Database constraint error mapping

Improves API usability and debugging consistency.

3.6 Search & Filtering

Implemented using **Spring Data JPA Specifications**.

Supported filters:

- Title
- Author
- Publication year
- Availability

Why Specifications?

- Dynamic query composition
 - Clean separation of search logic
 - High performance with indexed columns
 - Extensible for future filters
-

3.7 Soft Delete Strategy

Books are never physically removed.

Instead:

- `deleted = true` marks them inactive
- Borrowed books cannot be deleted
- Queries exclude deleted books

Rationale:

- Preserves historical data
 - Prevents accidental data loss
 - Maintains borrowing integrity
-

3.8 DTO & Mapping Layer

MapStruct is used for:

- Request DTO → Entity
- Entity → Response DTO

Benefits:

- Compile-time mapping
 - Zero runtime overhead
 - Reduced boilerplate
 - Clean separation between API and persistence
-

3.9 Database Schema & ORM

PostgreSQL

Chosen for:

- ACID compliance
- Strong indexing
- JSONB support
- Production reliability
- Superior concurrency handling

Flyway

Used for:

- Versioned migrations
- Controlled schema evolution
- Repeatable builds
- Safe production deployment

Hibernate

Used for:

- ORM mapping
- Lazy loading
- Relationship management
- Query generation

3.10 Testing Strategy

Unit Tests

Cover:

- Services
- Controllers
- Filters
- Mappers
- Exception handlers

Integration Tests

Using **Testcontainers** with real PostgreSQL.

Coverage

Measured with **JaCoCo**.

Static Analysis

Performed with **PMD**.

Formatting

Enforced with **Spotless**.

This ensures enterprise-grade code quality.

3.11 CI/CD Pipeline

GitHub Actions pipeline includes:

- Build
- Unit tests
- Integration tests
- Static analysis
- Coverage report
- Docker image build

Ensures consistent quality and production readiness.

3.12 DevOps & Deployment

Docker

Multi-stage builds produce optimized images.

Docker Compose

Orchestrates:

- PostgreSQL
- Application container

Start Script

Automates:

- Formatting
- Testing
- Coverage
- Static analysis
- Docker build
- Deployment

4. Architectural Design Decisions (Extremely Detailed)

4.1 Layered Architecture

Chosen for:

- Maintainability
- Testability
- Scalability
- Clear responsibility boundaries
- Enterprise alignment

Alternatives considered:

- Hexagonal architecture
- CQRS
- Event-driven architecture

Layered architecture was selected due to project scope and clarity.

4.2 Book as Individual Instance

Avoided “copies count” because:

- Each book needs its own borrowing history
- Availability must be tracked per instance
- Future item types require per-instance modeling
- Avoids ambiguity in concurrent borrowing scenarios

4.3 One Borrowing Record Per Transaction

Borrow → Return lifecycle stored in one record:

- Simplifies lifecycle tracking
- Preserves audit trails
- Avoids redundant data
- Supports analytics

4.4 PostgreSQL over MySQL

Reasons:

- Better JSON support

- Stronger indexing
 - More reliable at scale
 - Better concurrency handling
 - Rich extension ecosystem
-

4.5 Flyway + JPA Hybrid

Flyway handles schema; JPA validates mapping:

- Safe migrations
 - Full control
 - Production-ready
 - No accidental schema changes
-

4.6 Two Controllers

BookController and BorrowingController:

- Clear domain separation
 - Easier testing
 - Better API documentation
-

4.7 MapStruct

Chosen for:

- Compile-time mapping
 - Zero runtime overhead
 - Cleaner DTO handling
-

4.8 Soft Delete

Chosen to:

- Preserve history
 - Avoid accidental data loss
 - Maintain borrowing integrity
-

4.9 Specifications for Search

Chosen because:

- Dynamic filters
 - Composable queries
 - Clean separation of search logic
-

4.10 Testcontainers

Chosen for:

- Real database integration tests
 - Production-like environment
 - Reliable integration tests
-

4.11 CI/CD Pipeline

Ensures:

- Automated quality checks
- Consistent builds
- Deployment readiness

5. Future Work (Extremely Detailed)

Authentication & Authorization

- JWT
- Role-based access (LIBRARIAN, MEMBER)
- Method-level security

Analytics Dashboard

- Borrowing statistics
- Popular books
- Overdue reports

Email Notifications

- Borrowing confirmations
- Overdue reminders

Redis Caching

- Faster search results
- Reduced DB load

Reservation System

Allow users to reserve unavailable books.

Concurrency Handling

Prevent race conditions during simultaneous borrow attempts.

Expanded Inheritance Hierarchy

Add Magazine, Ebook, AudioBook.

Improved Documentation

More diagrams, sequence flows, and architectural notes.

6. Q&A Appendix – Full Professional Answers to All Handwritten Persian Questions

Below are all handwritten questions, rewritten clearly and answered at a senior-engineer level.

Q1 – Why use layered architecture?

Layered architecture enforces strict separation of concerns, enabling independent evolution of presentation, application, domain, and infrastructure layers. It improves maintainability, testability, and scalability while aligning with enterprise engineering standards.

Q2 – Why use Service layer instead of putting logic in controllers?

Controllers should remain thin and focus solely on HTTP orchestration. Business logic belongs in the Service layer to ensure reusability, testability, and adherence to SRP and DIP. This prevents controllers from becoming monolithic and unmaintainable.

Q3 – Why use validation (Bean Validation)?

Bean Validation ensures that invalid data never enters the system, reducing runtime errors and enforcing domain constraints at the boundary. It centralizes validation

logic, improves API reliability, and prevents inconsistent states.

Q4 – Why use filters and search specifications?

Specifications provide a composable, dynamic, and type-safe way to build queries. They eliminate the need for multiple custom repository methods and allow combining filters cleanly. This approach scales well as new search criteria are added.

Q5 – Why use MapStruct?

MapStruct generates mapping code at compile time, ensuring zero runtime overhead and eliminating boilerplate. It enforces type safety, improves maintainability, and keeps DTO/entity boundaries clean.

Q6 – Why use Global Exception Handler?

A centralized exception handler ensures consistent error responses, improves debugging, and prevents leaking internal details. It standardizes error formats and enhances API consumer experience.

Q7 – Why use PostgreSQL instead of MySQL?

PostgreSQL offers superior ACID compliance, indexing, concurrency handling, and JSONB support. It is more reliable for enterprise workloads and provides advanced features that benefit complex search and analytics.

Q8 – Why use Flyway instead of JPA auto-DDL?

Flyway provides deterministic, version-controlled migrations suitable for production environments. JPA auto-DDL is unpredictable and unsafe for real deployments. Flyway ensures schema consistency across environments.

Q9 – Why use soft delete instead of hard delete?

Soft delete preserves historical data, prevents accidental loss, and ensures borrowing records remain valid. It aligns with auditability requirements and avoids breaking referential integrity.

Q10 – Why treat each book as an instance instead of using “copies count”?

Instance-based modeling preserves individual borrowing history, supports future item types, and avoids ambiguity in availability tracking. It aligns with real library systems and simplifies concurrency handling.

Q11 – Why use abstract classes and inheritance?

Abstract classes allow defining shared behavior for future item types (Magazine, Ebook, AudioBook) while keeping Book implementation clean. This follows the Open/Closed Principle and supports future extensibility.

Q12 – Why use enums, interfaces, classes, and records?

Enums model fixed states; interfaces define contracts; classes represent domain entities; records provide immutable DTOs. This combination ensures clarity, correctness, and alignment with modern Java design patterns.

Q13 – Why use two controllers?

Borrowing and book management are distinct domains with different business rules. Splitting controllers improves clarity, reduces coupling, and simplifies testing and documentation.

Q14 – Why use Spotless, Jacoco, PMD?

Spotless enforces formatting; Jacoco ensures coverage; PMD detects bugs and code smells. Together they enforce code quality, maintainability, and consistency across the codebase.

Q15 – Why use Testcontainers?

Testcontainers provides real database integration tests using actual PostgreSQL instances. This ensures production-like reliability and eliminates issues caused by in-memory databases.

Q16 – Why use constructor injection?

Constructor injection ensures immutability, simplifies testing, and aligns with dependency inversion. It prevents null dependencies and improves code clarity.

Q17 – Why use Global Exception Handler for edge cases?

It ensures all errors are handled uniformly, prevents leaking internal details, and provides structured responses for unexpected conditions.

Q18 – Why design borrowing as a single record?

A single record simplifies lifecycle tracking, preserves audit trails, and avoids redundant data. It aligns with real library workflows and supports future analytics.

Q19 – Why use JPA annotations like @Entity, @Id, @OneToMany?

They provide declarative ORM mapping, simplify relationship management, and ensure type-safe persistence. This reduces boilerplate and improves maintainability.

Q20 – Why use CI/CD pipeline?

CI/CD automates testing, analysis, and deployment, ensuring consistent quality and reducing manual errors. It enforces engineering discipline and accelerates delivery.

7. Conclusion

The Library Management System is a fully engineered, production-ready backend application demonstrating mastery of modern Java backend development. It follows enterprise architecture principles, SOLID design, clean layering, robust DevOps practices, and comprehensive testing.

The system is scalable, maintainable, and extensible, with clear separation of concerns and robust business logic. The architectural decisions reflect thoughtful engineering, balancing simplicity with future-proofing.

This rewritten report provides a complete, extremely detailed technical overview suitable for senior engineering review, enterprise documentation, or professional interviews.